Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №4
на тему
«ЗАЩИТА ОТ АТАКИ ПРИ УСТАНОВКЕ TCP-СОЕДИНЕНИЯ И ПРОТОКОЛОВ ПРИКЛАДНОГО УРОВНЯ»

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

_____
(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Ограничение частоты запросов (Rate Limiting) 4
2.2 Доказательство выполнения работы (Proof of Work) 5
2.3 Тестирование системы 5
Заключение 6
Приложение А (обязательное) Листинг программного кода 7

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение и реализация методов защиты от сетевых атак, направленных на исчерпание ресурсов при установке TCP-соединений и на прикладном уровне. Задачи включают изучение механизмов ограничения частоты запросов (Rate Limiting) и алгоритмов доказательства выполнения работы (Proof of Work), проектирование и программную реализацию защищённого сервера и клиента, а также тестирование устойчивости системы к имитируемой атаке.

Конечным результатом лабораторной работы является программная система, состоящая из сервера с многоуровневой защитой, легитимного клиента и атакующего скрипта, демонстрирующая эффективность применённых мер.

2 ХОД РАБОТЫ

В ходе выполнения работы была разработана программная система на языке Python, реализующая механизмы защиты от DoS-атак. Система включает в себя три основных компонента:

1 Сервер, осуществляющий мониторинг входящих соединений, ограничение частоты запросов по IP-адресу и проверку доказательства выполнения работы (PoW) перед предоставлением доступа к ресурсу.

2 Клиент, способный корректно взаимодействовать с протоколом защиты, включая решение криптографических задач (PoW), выдаваемых сервером.

3 Атакующий (скрипт), имитирующий многопоточную атаку на сервер с целью обхода или исчерпания его ресурсов.

Процесс установления защищённого соединения включает следующие этапы:

1 Предварительная проверка (Rate Limiting).

2 Выдача и решение криптографической задачи (PoW).

3 Верификация решения и предоставление доступа.

2.1 Ограничение частоты запросов (Rate Limiting)

Первым эшелоном защиты является механизм ограничения частоты запросов. Для каждого входящего IP-адреса сервер хранит историю меток времени последних подключений. Если количество запросов от одного IP за определённое временное окно (например, 10 секунд) превышает заданный лимит, сервер немедленно разрывает соединение с ошибкой. Данный метод эффективен против простых атак перебором и примитивных попыток затопления трафиком (flooding) с одного адреса. Фрагмент журнала событий сервера при попытке атаки представлен на рисунке 2.1.

```
=== Server started on port 65432 ===
=== Difficulty: 4, limit: 3 in 10s ===
[*] New connection: ('127.0.0.1', 55718)
[+] 127.0.0.1 successfully solved PoW!
[*] New connection: ('127.0.0.1', 45294)
[*] New connection: ('127.0.0.1', 45300)
[*] New connection: ('127.0.0.1', 45302)
[*] New connection: ('127.0.0.1', 45316)
[!] Rate limit for 127.0.0.1 exceeded
[*] New connection: ('127.0.0.1', 45320)
[!] Rate limit for 127.0.0.1 exceeded
[*] New connection: ('127.0.0.1', 45330)
[!] Rate limit for 127.0.0.1 exceeded
[*] New connection: ('127.0.0.1', 45344)
```

Рисунок 2.1 – Логи сервера при ограничении частоты запросов

2.2 Доказательство выполнения работы (Proof of Work)

Для защиты от более сложных атак и распределённых запросов используется механизм PoW на основе алгоритма SHA-256. Процесс реализован следующим образом:

1 Сервер генерирует случайный токен (challenge) и отправляет его клиенту вместе с параметром сложности (количеством требуемых нулевых битов в начале хеша).

2 Клиент должен найти такое число (nonce), чтобы хеш от конкатенации челленджа и этого числа удовлетворял условию сложности.

3 Сервер проверяет полученное решение за константное время, в то время как поиск решения требует от клиента значительных вычислительных затрат.

Процесс решения задачи клиентом представлен на рисунке 2.2.

```
→  acryoz lab4 git:(main ?:1 ✗) python client.py
[*] Solving challenge of difficulty 4...
[+] Solved in 0.0155 s. Nonce: 38178
[*] Server response: SUCCESS: Welcome to the protected resource!
```

Рисунок 2.2 – Процесс решения криптографической задачи клиентом

Использование PoW вынуждает атакующего тратить вычислительные ресурсы на каждое соединение, что делает атаку экономически невыгодной или технически невозможной при больших масштабах.

2.3 Тестирование системы

Для проверки эффективности защиты был использован скрипт-атакующий, запускающий 20 параллельных потоков для агрессивного подключения к серверу.

– Механизм Rate Limiting успешно заблокировал попытки превышения лимита запросов с одного IP-адреса.

– Механизм PoW обеспечил защиту от быстрой обработки запросов, так как каждое соединение требовало времени на вычисления.

– Легитимный клиент, выполняющий все требования протокола, успешно получал доступ к ресурсу после решения задачи.

Исходный код программных модулей представлен в приложении А.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и практически реализованы ключевые методы защиты сетевых сервисов от атак при установке соединений. Реализованная комбинация Rate Limiting и Proof of Work показала высокую эффективность в противодействии попыткам исчерпания ресурсов сервера.

На первом этапе был внедрён механизм ограничения частоты запросов, который позволил отсечь избыточную нагрузку от отдельных узлов сети. Второй этап — внедрение PoW — позволил защитить прикладной уровень от автоматизированных запросов, перекладывая вычислительную нагрузку на сторону клиента. Это создаёт асимметрию затрат: проверка решения сервером практически мгновенна, тогда как генерация решения требует от клиента реальных затрат времени и энергии.

Результаты тестирования подтвердили работоспособность системы. Атаки, имитируемые многопоточным скриптом, были успешно нейтрализованы, при этом сохранилась доступность сервиса для добросовестных пользователей. Все поставленные цели были достигнуты, а реализованные механизмы продемонстрировали надёжность и масштабируемость в рамках учебной задачи.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Реализация защищённого сервера

```
import socket
import hashlib
import secrets
import threading
import time
from collections import defaultdict

PORT = 65432
DIFFICULTY = 4
CONN_TIMEOUT = 5.0
RATE_LIMIT_WINDOW = 10
MAX_REQ_PER_WINDOW = 3

request_history = defaultdict(list)
history_lock = threading.Lock()

def is_rate_limited(ip):
    now = time.time()
    with history_lock:
        request_history[ip] = [t for t in request_history[ip] if now - t <
RATE_LIMIT_WINDOW]
        if len(request_history[ip]) >= MAX_REQ_PER_WINDOW:
            return True
        request_history[ip].append(now)
        return False

def verify_pow(challenge, nonce):
    prefix = '0' * DIFFICULTY
    attempt = f"{challenge}{nonce}".encode()
    result = hashlib.sha256(attempt).hexdigest()
    return result.startswith(prefix)

def handle_client(conn, addr):
    ip = addr[0]
    try:
        if is_rate_limited(ip):
            conn.sendall(b"ERROR: Rate limit exceeded. Try again later.")
            return

        challenge = secrets.token_hex(16)
        conn.sendall(f"CHALLENGE:{challenge}:{DIFFICULTY}".encode())

        conn.settimeout(CONN_TIMEOUT)
        data = conn.recv(1024).decode().strip()

        if data.startswith("SOLVE:"):
            nonce = data.split(":")[1]
            if verify_pow(challenge, nonce):
                conn.sendall(b"SUCCESS: Welcome to the protected resource!")
            else:
                conn.sendall(b"ERROR: Invalid PoW solution.")
    finally:
        conn.close()

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    server.bind(('0.0.0.0', PORT))
    server.listen(100)
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()
```

Листинг А.2 – Реализация легитимного клиента

```
import socket
import hashlib
import time

def solve_pow(challenge, difficulty):
    prefix = '0' * int(difficulty)
    nonce = 0
    while True:
        res = hashlib.sha256(f"{challenge}{nonce}".encode()).hexdigest()
        if res.startswith(prefix):
            return str(nonce)
        nonce += 1

def run_client():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect(('127.0.0.1', 65432))
    resp = s.recv(1024).decode().strip()
    if "CHALLENGE" in resp:
        parts = resp.split(":")
        nonce = solve_pow(parts[1], parts[2])
        s.sendall(f"SOLVE:{nonce}".encode())
        print(f"Server response: {s.recv(1024).decode()}")
    s.close()
```

Листинг А.3 – Реализация атакующего скрипта

```
import socket
import threading
import time

def attack_once(thread_id):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(('127.0.0.1', 65432))
        resp = s.recv(1024).decode().strip()
        time.sleep(1)
    finally:
        s.close()

def run_attack():
    threads = []
    for i in range(20):
        t = threading.Thread(target=attack_once, args=(i,))
        threads.append(t)
        t.start()
        time.sleep(0.05)
    for t in threads: t.join()
```